1 Lab Overview

Lab Objectives

By the end of this lab, you will be able to:

- Understand the LBM event-driven architecture
- Implement event handlers for LoRaWAN events
- Handle JOINED, TXDONE, DOWNDATA events
- Process event queues correctly
- Track application state based on modem events

Prerequisites

Required Knowledge:

- Completion of Lab 1.1 (Basic LoRaWAN Join)
- Understanding of event-driven programming
- Familiarity with LBM API

Hardware Required:

- Xiao nRF54L15 + LR1120 board
- USB cable
- LoRaWAN gateway in range

2 Lab Duration

Estimated Time: 60 minutes

3 Background

3.1 LBM Event-Driven Architecture

LoRa Basics Modem (LBM) uses an event-driven architecture where the application responds to asynchronous events from the modem. This design:

- Decouples application logic from modem timing
- Enables low-power operation (sleep between events)
- Simplifies handling of asynchronous LoRaWAN operations
- Supports multiple concurrent operations

3.2 Event Types

Common LBM events include:

Event	Description
JOINED	Device successfully joined network
TXDONE	Uplink transmission completed
DOWNDATA	Downlink data received
UPLOAD_DONE	File upload completed (FUOTA)
LINK_CHECK	Link check result available
ALARM	Scheduled alarm triggered

4 Lab Instructions

4.1 Step 1: Initialize LBM Stack

Create a new application that uses LBM instead of Zephyr's native LoRaWAN stack.

4.2 Step 2: Implement Event Handler

Create an event handler function that processes all modem events in a loop.

4.3 Step 3: Handle JOIN Event

When JOINED event is received, initiate first uplink transmission.

4.4 Step 4: Handle TXDONE Event

Track transmission status (confirmed/unconfirmed, ACK received, etc.).

4.5 Step 5: Handle DOWNDATA Event

Process received downlink messages and extract payload.

4.6 Step 6: Build and Test

```
west build -b xiao_nrf54l15 .
west flash
```

5 Solution

Solution

5.1 Complete Implementation

```
#include <zephyr/kernel.h>
#include <zephyr/logging/log.h>
#include <smtc_modem_api.h>
#include <smtc_board.h>

LOG_MODULE_REGISTER(lab_2_1, LOG_LEVEL_INF);

/* LoRaWAN Credentials */
#define LORAWAN_DEV_EUI    { 0x00, 0x00, 0x00, 0x00, \
                             0x00, 0x00, 0x00, 0x01 }
```

```c
#define LORAWAN_JOIN_EUI    { 0x00, 0x00, 0x00, 0x00, \
                              0x00, 0x00, 0x00, 0x00 }
#define LORAWAN_APP_KEY     { 0x00, 0x00, 0x00, 0x00, \
                              0x00, 0x00, 0x00, 0x00, \
                              0x00, 0x00, 0x00, 0x00, \
                              0x00, 0x00, 0x00, 0x00 }

/* Application state */
static struct {
    bool joined;
    uint32_t uplink_count;
    uint32_t downlink_count;
    int16_t last_rssi;
    int8_t last_snr;
} app_state = {0};

/**
 * @brief Handle JOINED event
 */
static void handle_joined_event(void)
{
    LOG_INF("==========================================");
    LOG_INF("JOINED EVENT: Network joined successfully");
    LOG_INF("==========================================");

    app_state.joined = true;

    /* Get and display DevAddr */
    uint32_t dev_addr;
    smtc_modem_get_dev_addr(0, &dev_addr);
    LOG_INF("DevAddr: 0x%08X", dev_addr);

    /* Send first uplink */
    uint8_t payload[] = "Hello from LBM!";
    smtc_modem_request_uplink(0, 2, false,
                              payload, sizeof(payload));
    LOG_INF("First uplink queued");
}

/**
 * @brief Handle TXDONE event
 */
static void handle_txdone_event(smtc_modem_event_t *event)
{
    app_state.uplink_count++;

    LOG_INF("TXDONE EVENT #%u:", app_state.uplink_count);
    LOG_INF("  Status: %s",
            event->event_data.txdone.status ==
            SMTC_MODEM_EVENT_TXDONE_CONFIRMED
                ? "CONFIRMED" : "UNCONFIRMED");

    if (event->event_data.txdone.status ==
        SMTC_MODEM_EVENT_TXDONE_CONFIRMED) {
        LOG_INF("  ACK received");
    }
```

```c

    /* Schedule next uplink */
    k_work_schedule(&uplink_work, K_SECONDS(60));
}

/**
 * @brief Handle DOWNDATA event
 */
static void handle_downdata_event(smtc_modem_event_t *event)
{
    uint8_t rx_payload[255];
    uint8_t rx_payload_size;
    smtc_modem_dl_metadata_t rx_metadata;

    app_state.downlink_count++;

    LOG_INF("DOWNDATA EVENT #%u:", app_state.downlink_count);

    /* Get downlink data */
    smtc_modem_get_downlink_data(
        0, rx_payload, &rx_payload_size,
        &rx_metadata);

    LOG_INF("  Port: %d", rx_metadata.fport);
    LOG_INF("  RSSI: %d dBm", rx_metadata.rssi);
    LOG_INF("  SNR: %d dB", rx_metadata.snr);
    LOG_INF("  Length: %d bytes", rx_payload_size);

    if (rx_payload_size > 0) {
        LOG_HEXDUMP_INF(rx_payload, rx_payload_size,
                        "Payload");

        /* Store link quality */
        app_state.last_rssi = rx_metadata.rssi;
        app_state.last_snr = rx_metadata.snr;
    }
}

/**
 * @brief Main event processing loop
 */
static void process_modem_events(void)
{
    smtc_modem_event_t event;
    uint8_t pending_events;

    /* Process all pending events */
    while (smtc_modem_get_event(&event, &pending_events)
           == SMTC_MODEM_RC_OK) {

        LOG_DBG("Event received: type=%d, pending=%d",
                event.event_type, pending_events);

        switch (event.event_type) {
        case SMTC_MODEM_EVENT_RESET:
            LOG_INF("RESET EVENT: Modem reset");
```

```
            break;

        case SMTC_MODEM_EVENT_ALARM:
            LOG_INF("ALARM EVENT: Timer expired");
            break;

        case SMTC_MODEM_EVENT_JOINED:
            handle_joined_event();
            break;

        case SMTC_MODEM_EVENT_TXDONE:
            handle_txdone_event(&event);
            break;

        case SMTC_MODEM_EVENT_DOWNDATA:
            handle_downdata_event(&event);
            break;

        case SMTC_MODEM_EVENT_JOINFAIL:
            LOG_ERR("JOIN FAIL EVENT");
            /* Retry join */
            k_work_schedule(&join_work, K_SECONDS(10));
            break;

        case SMTC_MODEM_EVENT_LINK_CHECK:
            LOG_INF("LINK CHECK EVENT:");
            LOG_INF("  Margin: %d dB",
                    event.event_data.link_check.margin);
            LOG_INF("  Gateways: %d",
                    event.event_data.link_check.gw_cnt);
            break;

        case SMTC_MODEM_EVENT_MUTE:
            LOG_WRN("MUTE EVENT: Device muted by network");
            break;

        default:
            LOG_WRN("Unhandled event: %d",
                    event.event_type);
            break;
        }
    }
}

/* Work queue for periodic uplinks */
static void uplink_work_handler(struct k_work *work)
{
    if (!app_state.joined) {
        LOG_WRN("Not joined, skipping uplink");
        return;
    }

    uint8_t payload[32];
    int len = snprintf(payload, sizeof(payload),
                       "Uplink #%u", app_state.uplink_count + 1);

```

```
    smtc_modem_request_uplink(0, 2, false, payload, len);
    LOG_INF("Uplink queued: %s", payload);
}

K_WORK_DELAYABLE_DEFINE(uplink_work, uplink_work_handler);

/* Work queue for join retry */
static void join_work_handler(struct k_work *work)
{
    LOG_INF("Retrying join...");
    smtc_modem_join_network(0);
}

K_WORK_DELAYABLE_DEFINE(join_work, join_work_handler);

int main(void)
{
    int ret;

    LOG_INF("================================================");
    LOG_INF("Lab 2.1: Event-Driven Programming with LBM");
    LOG_INF("================================================");

    /* Initialize board-specific hardware */
    smtc_board_init();

    /* Initialize modem */
    ret = smtc_modem_init();
    if (ret != SMTC_MODEM_RC_OK) {
        LOG_ERR("Modem init failed: %d", ret);
        return -1;
    }

    LOG_INF("Modem initialized");

    /* Set credentials */
    uint8_t dev_eui[] = LORAWAN_DEV_EUI;
    uint8_t join_eui[] = LORAWAN_JOIN_EUI;
    uint8_t app_key[] = LORAWAN_APP_KEY;

    smtc_modem_set_deveui(0, dev_eui);
    smtc_modem_set_joineui(0, join_eui);
    smtc_modem_set_nwkkey(0, app_key);

    /* Set region */
    smtc_modem_set_region(0, SMTC_MODEM_REGION_EU_868);

    LOG_INF("Credentials configured");

    /* Start join procedure */
    LOG_INF("Starting OTAA join...");
    ret = smtc_modem_join_network(0);
    if (ret != SMTC_MODEM_RC_OK) {
        LOG_ERR("Join failed: %d", ret);
        return -1;
    }
```

```
    /* Main event loop */
    LOG_INF("Entering event loop");

    while (1) {
        /* Process modem events */
        process_modem_events();

        /* Run modem engine */
        smtc_modem_run_engine();

        /* Sleep until next event */
        k_sleep(K_MSEC(100));
    }

    return 0;
}
```

5.2 CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20.0)
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
project(lab_2_1)

target_sources(app PRIVATE src/main.c)

# Include LBM library
target_link_libraries(app PRIVATE lora_basics_modem)
```

5.3 prj.conf

```
# LBM Configuration
CONFIG_LORA_BASICS_MODEM=y
CONFIG_LBM_REGION_EU868=y

# Logging
CONFIG_LOG=y
CONFIG_LBM_LOG_LEVEL_DBG=y
CONFIG_LOG_BUFFER_SIZE=4096

# Stack sizes
CONFIG_MAIN_STACK_SIZE=4096
CONFIG_SYSTEM_WORKQUEUE_STACK_SIZE=2048
```

6 Expected Output

```
=============================================
Lab 2.1: Event-Driven Programming with LBM
=============================================
[00:00:00.100] <inf> lab_2_1: Modem initialized
[00:00:00.105] <inf> lab_2_1: Credentials configured
[00:00:00.110] <inf> lab_2_1: Starting OTAA join...
```

```
[00:00:00.115] <inf> lab_2_1: Entering event loop
[00:00:05.234] <inf> lab_2_1: ==================
[00:00:05.235] <inf> lab_2_1: JOINED EVENT
[00:00:05.236] <inf> lab_2_1: ==================
[00:00:05.237] <inf> lab_2_1: DevAddr: 0x260B1234
[00:00:05.238] <inf> lab_2_1: First uplink queued
[00:00:06.456] <inf> lab_2_1: TXDONE EVENT #1:
[00:00:06.457] <inf> lab_2_1:   Status: UNCONFIRMED
[00:01:06.457] <inf> lab_2_1: Uplink queued: Uplink #2
[00:01:07.678] <inf> lab_2_1: TXDONE EVENT #2:
[00:01:07.679] <inf> lab_2_1:   Status: UNCONFIRMED
[00:01:08.123] <inf> lab_2_1: DOWNDATA EVENT #1:
[00:01:08.124] <inf> lab_2_1:   Port: 2
[00:01:08.125] <inf> lab_2_1:   RSSI: -45 dBm
[00:01:08.126] <inf> lab_2_1:   SNR: 10 dB
[00:01:08.127] <inf> lab_2_1:   Length: 4 bytes
```

7 Deliverables

1. **Source Code** with event handlers for all required events
2. **Serial Log** showing successful event processing
3. **Event Timing Analysis**
   - Time from join to first TXDONE
   - Average time between uplinks
   - Response time to downlinks
4. **Lab Report** describing event flow and state transitions

8 Additional Challenges

8.1 Challenge 1: Event Statistics

Track statistics for each event type:

- Count of each event received
- Average time between events
- Display summary every 10 minutes

8.2 Challenge 2: Custom Event Actions

Implement custom actions based on link quality:

- If RSSI $<$ -100 dBm, increase SF
- If SNR $<$ 0 dB, request link check
- Log quality trends